

Tutorial 12

Exercise 1: Decrementing counters

1. Let us add a decrement operation `DEC` to the k -bit counter example from the lecture, which is implemented analogously to the increment operation. In particular, the cost of one decrement is the number of bits inspected and/or updated.

Show that a sequence of n operations on an initially zero counter can cost at least $n \cdot k$ time.

2. Let us add a reset operation `RESET` to the k -bit counter example from the lecture (i.e., without decrement). Again, the cost of one reset is the number of bits inspected and/or updated.

Implement a counter as an array of bits so that any sequence of n increments and resets takes $\mathcal{O}(n)$ time on an initially zero counter. Show your implementation of the operations and the analysis.

Hint: Keep a pointer to the highest-order 1.

Solution:

1. Decrementing a counter holding the value zero (all bits are zero) yields a counter with all bits being one. Dually, incrementing a counter with all bits being one yields a counter with all bits being zero. Hence, when starting at the zero counter, a sequence of n alternating decrements and increments (starting with a decrement) flips every bit during each operations, i.e., $n \cdot k$ bit flips. As each bitflip takes constant time, the sequence takes overall time of at least $(n \cdot k)$.
2. We add a new variable p which keeps that of the maximal i such that $A[i] = 1$, initialized with zero. Then, we implement increment and reset as follows:

```

1 def inc2(A, p):
2     i = 0
3     while(i < length(A) and A[i] == 1):
4         A[i] = 0
5         i = i + 1
6     if i < length(A):
7         A[i] = 1
8         p = max(p, i)
9     else:
10        p = -1
11
12 def reset(A, p):
13     i = 0
14     while(i <= p):
15         A[i] = 0
16     p = -1

```

As in the lecture/book, we assume that the actual cost of a bitflip is 1. In addition, we assume the cost of updating p is 1 as well.

Setting and resetting of bits by `INC` will work exactly as for the original counter in the lecture (also, read the corresponding section in the book): Setting a bit to 1, the credit is increased by 1, and that credit is used to reset the bit during incrementing. In addition, we increase the credit to later update p (if p doesn't increase, we can just waste that credit, it won't be needed.)

Since RESET manipulates bits at positions only up to p , and since each bit up to there must have become the highest-order 1 at some time before the highest-order 1 got up to p , every bit seen by RESET has some nonzero credit left. So the zeroing of bits of A by RESET can be completely paid for by the credit stored on the bits.

We just need cost 1 to pay for resetting max. Thus, a charge of 4 for each INC and 1 for each RESET is sufficient, so that the sequence of n INC and RESET operations takes $\mathcal{O}(n)$ time.

Exercise 2: Applying amortized analysis

Consider a sequence of n operations on some data structure in which the i -th operation costs i if i is an exact power of two (i.e., $i \in \{1, 2, 4, 8, 16, \dots\}$) and 1 otherwise.

1. Use aggregate analysis to determine the amortized cost per operation of such a sequence (for an arbitrary n).
2. Use the accounting method to determine the amortized cost per operation of such a sequence (for an arbitrary n).
3. Challenge: Use the potential method to determine the amortized cost per operation of such a sequence (for an arbitrary n).

Solution:

1. The actual cost c_i of operation i is

$$c_i = \begin{cases} i & \text{if } i \text{ is a power of 2} \\ 1 & \text{otherwise.} \end{cases}$$

Hence, we have

$$\sum_{i=1}^n c_i \leq n + \sum_{i=1}^{\log_2(n)} 2^i \leq n + 2n = 3n.$$

Here, we used the facts $\sum_{i=1}^n 2^i = 2^{n+1} - 2$ and $2^{\log_2(n)+1} = 2n$.

Hence, the cost of a sequence of n operations is at most $3n$, i.e., the amortized cost of each operation is at most 3.

2. We charge cost 3 for each operation. Then, the credit after n operations is $3n - (\sum_{i=1}^n c_i)$, which is always nonnegative as we have seen in Item 1. that $\sum_{i=1}^n c_i \leq 3n$.

Hence, since the amortized cost of each operation is constant and the credit is never negative, the total cost of n operations is $\mathcal{O}(n)$.

3. Let D_i denote the data structure before the i -th operation.

Consider $i \geq 1$ and let p_i be the largest power of 2 that is smaller or equal to i and let $\Delta_i = i - p_i$, which is always nonnegative. In particular, Δ_i is zero if and only if i is a power of 2.

We define the function Φ as $\Phi_0 = 0$ and $\Phi(D_i) = 2\Delta_i$. This satisfies the requirements of a potential function, as argued above.

We now compute the amortized cost of each operation by considering two cases:

- If $\Delta_i = 0$, then i is a power of two (say $i = 2^j$), and we have $i - 1 = 2^j - 1$. Hence, $p_i = 2^{j-1}$ and

$$\Delta_{i-1} = (i - 1) - 2^{j-1} = 2^j - 1 - 2^{j-1} = 2 \cdot 2^{j-1} - 2^{j-1} - 1 = 2^{j-1} - 1.$$

Hence, the amortized cost of the i -th operation is

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = i - 2(2^{j-1} - 1) = i - 2^j + 2,$$

which is equal to 2, as $i = 2^j$.

- If $\Delta_i \neq 0$, then

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = 1 + 2\Delta_i - 2\Delta_{i-1}.$$

Now, as i is not a power of 2, we have $p_i = p_{i-1}$, which implies $\Delta_i = \Delta_{i-1} + 1$. Hence,

$$1 + 2\Delta_i - 2\Delta_{i-1} = 1 + 2(\Delta_{i-1} + 1) - 2\Delta_{i-1} + 1 = 3.$$

Hence, the amortized cost of each operation is at most 3. Thus, the amortized cost of a sequence of n operations is at most $3n$, which bounds the actual cost of the sequence.

Exercise 3: Bounded stacks

You perform a sequence of `PUSH` and `POP` operations on a stack whose size never exceeds k . After every k operations, a copy of the entire stack is made automatically, for backup purposes.

Show that the cost of n stack operations, including the copying of the stack, is $\mathcal{O}(n)$ by assigning suitable amortized costs to the various stack operations.

Solution:

The cost of a copy operation is proportional to the stack height of the stack to be copied, which is bounded by k .

We assign amortized cost 3 to the `PUSH` operation, 1 to the `POP` operation, and 0 to the copy operation.

Then, one can show by induction that the credit after n operations is at least the stack height of the current stack plus the number of operations since the last copy. Hence, when a copy is executed there have been k operations since the last copy, which gained enough credit to pay for the copy.

Exercise 4: Potential functions

Suppose you have a potential function Φ such that $\Phi(D_i) \geq \Phi(D_0)$ for all i , but $\Phi(D_0) \neq 0$. Show that there is a potential function Φ' such that $\Phi'(D_0) = 0$, $\Phi'(D_i) \geq 0$ for all i , and the amortized costs using Φ' are the same as the amortized costs using Φ .

Solution:

Let $\Delta = \Phi(D_0)$. We define the function Φ' by $\Phi'(D_i) = \Phi(D_i) - \Delta$ and claim that Φ' has the desired properties, i.e., it is a potential function such that $\Phi'(D_0) = 0$, $\Phi'(D_i) \geq 0$ for all i , and the amortized costs using Φ' are the same as the amortized costs using Φ .

- We have $\Phi'(D_0) = \Phi(D_0) - \Phi(D_0) = 0$ as required.
- We have $\Phi'(D_i) = \Phi(D_i) - \Phi(D_0) \geq 0$, as we have $\Phi(D_i) \geq \Phi(D_0)$ for all i by assumption on Φ .
- Let c_i be an operation. As Φ is a potential function, the amortized cost $\hat{c}_i$ of c_i w.r.t. Φ is

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}).$$

Now, the amortized cost $\hat{c}'_i$ of c_i w.r.t. Φ' is defined as

$$\hat{c}'_i = c_i + \Phi'(D_i) - \Phi'(D_{i-1}).$$

By definition of Φ' , we have

$$c_i + \Phi'(D_i) - \Phi'(D_{i-1}) = c_i + (\Phi(D_i) - \Phi(D_0)) - (\Phi(D_{i-1}) - \Phi(D_0)) = c_i + \Phi(D_i) - \Phi(D_{i-1}) = \hat{c}_i,$$

i.e., the amortized costs of c_i w.r.t. Φ and Φ' are equal as required.